

Relatório Técnico - Micro serviço para geração de relatórios
Case técnico

Github:

github.com/Guilherme-Fumagali/micro-servico-pedidos

Docker Hub:

hub.docker.com/r/gfumagali/micro-servico-pedidos

Guilherme Fumagali Marques

SÃO PAULO
2024

Sumário

Sumário.....	2
Especificação.....	3
Interface de comunicação.....	3
Consumo de mensagens.....	3
Pontos de acesso à API.....	4
Implementação.....	4
Tecnologias utilizadas.....	4
Ambiente de desenvolvimento.....	6
Diagrama de implantação.....	7
Diagrama da arquitetura.....	8
Modelagem da Base de Dados.....	9
Plano de trabalho.....	11
Testes funcionais.....	11
Valor total dos pedidos.....	12
Quantidade de pedidos por cliente.....	14
Lista de pedidos por cliente.....	14

Especificação

A demanda apresentada requisita um micro serviço para realizar um processamento intermediário entre um sistema de vendas e um sistema de visualização de relatórios. A comunicação pré definida entre os módulos esperam que:

- 1) O sistema de vendas irá publicar em um serviço de mensageria de objetos seguindo o padrão de notação JSON em esquema padronizado;
- 2) O micro serviço intermediário deve consumir esses dados e extrair/atualizar as informações de preço total do pedido, lista e quantidade de pedidos realizados por cliente;
- 3) O sistema de relatórios acessa os dados por meio de requisições HTTP a uma interface REST servida pelo micro serviço.

Interface de comunicação

Essa seção define os formatos esperados de troca de mensagens entre o micro serviço e os módulos que compõem o sistema. Considerando tanto os dados de entrada, consumidos da fila, quanto os dados de saída, requisitados na API.

Consumo de mensagens

Quando executado, o micro serviço se conecta a um tópico da fila configurável, criando-a se necessário. Cada mensagem recebida é submetida a uma validação para garantir que o objeto atenda a determinadas restrições, definidas logo abaixo:

```
Unset
{
  "codigoPedido": "long",          // Positivo ou zero
  "codigoCliente": "long",         // Positivo ou zero
  "itens": [                       // Pelo menos um item
    {
      "produto": "string",         // Pelo menos um caractere
      "quantidade": "long",        // Positivo maior que zero
      "preco": "double"            // Positivo ou zero, máximo de dez
                                   // dígitos e duas casas decimais
    },
  ]
}
```

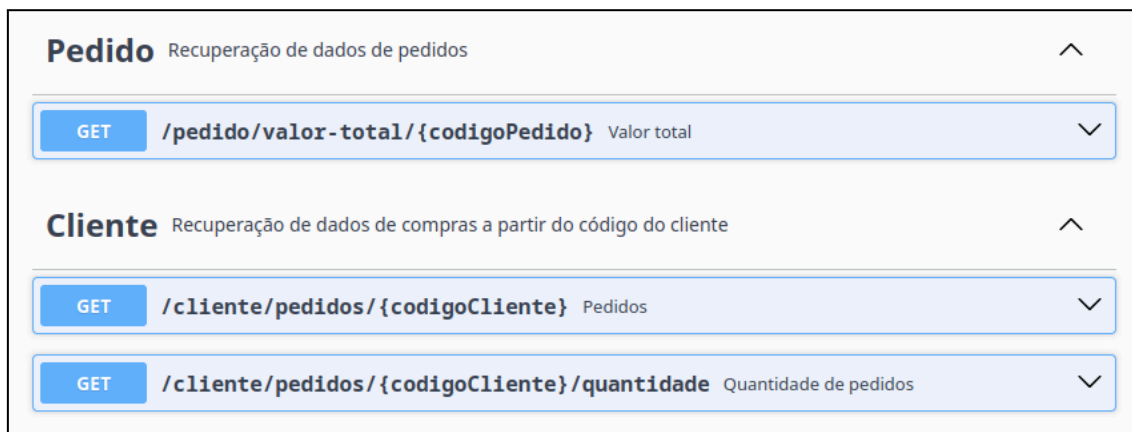
Caso qualquer uma dessas restrições falhe, ou a cadeia binária recebida não apresente uma decodificação válida do exato esquema apresentado, a mensagem é descartada e esse evento é registrado no *log* da aplicação. A execução do micro serviço continua sem interrupções.

Em relação às regras de negócios, uma mensagem com um *codigoPedido* repetido é considerada válida apenas se o registro anterior e o recebido atual pertencem ao mesmo cliente, o qual é identificado pelo *codigoCliente*. Nesse caso, a

sobrescrita do registro anterior é executada. Em qualquer outro cenário, a mensagem é descartada e o evento é registrado no *log* da aplicação. Em ambas ocasiões a aplicação segue funcionando normalmente.

Pontos de acesso à API

A aplicação expõe apenas pontos de acesso (*endpoints*) de leitura (verbo GET), acessíveis por meio de requisições HTTP destinadas a uma porta configurável na máquina. A Figura 1 lista as URLs desses *endpoints*. Para obter detalhes sobre códigos HTTP, MIME types, parâmetros de URL, filtragem e formato de retorno, consulte a documentação disponível em: <https://editor.swagger.io/?url=https://raw.githubusercontent.com/Guilherme-Fumagali/micro-servico-pedidos/main/static/openapi.json>.

A table with three rows representing API endpoints. The first row is for 'Pedido' (Recuperação de dados de pedidos) with a GET method and endpoint '/pedido/valor-total/{codigoPedido}' returning 'Valor total'. The second row is for 'Cliente' (Recuperação de dados de compras a partir do código do cliente) with a GET method and endpoint '/cliente/pedidos/{codigoCliente}' returning 'Pedidos'. The third row is also for 'Cliente' with a GET method and endpoint '/cliente/pedidos/{codigoCliente}/quantidade' returning 'Quantidade de pedidos'. Each row has a dropdown arrow on the right.

Pedido	Recuperação de dados de pedidos	^
GET	/pedido/valor-total/{codigoPedido}	Valor total
Cliente	Recuperação de dados de compras a partir do código do cliente	^
GET	/cliente/pedidos/{codigoCliente}	Pedidos
GET	/cliente/pedidos/{codigoCliente}/quantidade	Quantidade de pedidos

Figura 1. Especificação da API

Implementação

O código-fonte da aplicação pode ser acessado em <https://github.com/Guilherme-Fumagali/micro-servico-pedidos>. Em uma visão arquitetural de alto nível, o modo de implementação e tecnologias utilizadas seguem os fatores descrito em <https://12factor.net>, que explicita uma série de boas práticas para um software portátil, no sentido de ter uma implantação (*deploy*) facilitada independente do ambiente.

Tecnologias utilizadas

O código-fonte da aplicação é escrito em Java e utiliza o [framework Spring](#). O Spring fornece suporte para mapeamento objeto-relacional (ORM) para acesso a bancos de dados, configuração de container web para lidar com requisições, controle de injeção de dependências para reduzir o acoplamento entre classes, entre outras funcionalidades que facilitam a construção de microsserviços e aplicações Java em geral. Em relação ao documento dos Doze Fatores, o *framework* abrange aspectos

como configurações (III), vínculo de porta (VII), concorrência (VIII), descartabilidade (IX) e logs (XI).

Seguindo para a declaração de dependências, a ferramenta de *build* utilizada foi o [Gradle](#), uma ferramenta open-source que atua na automatização de *downloads* de dependências declaradas e do processo de geração de arquivos *.jar*, binário da aplicação executado por uma JVM Java. Toda especificação é inserida em um arquivo nomeado de *build.gradle*, que contém metadados sobre a versão 21 do Java utilizada e todas as bibliotecas das quais o software depende. Essas bibliotecas são:

- **[Starter amqp](#)**: provém a interface de conexão com o serviço de mensageria RabbitMQ;
- **[Starter Mongoddb](#)**: implementa ORM e métodos para criação e envio de queries para o servidor Mongo (banco de dados);
- **[Starter Rest](#)**: funções e anotações utilizadas para definir a API de leitura implementada;
- **[Starter Validation](#)**: métodos que atuam na validação de objetos de entrada e saída da aplicação, como os dados das mensagens e os parâmetros de filtragem no *endpoint* de listar pedidos do usuário;
- **[Starter Test](#)**: biblioteca utilizada para desenvolver os testes unitários da aplicação, com métodos para a criação de *mock* e validação;
- **[Springdoc OpenAPI](#)**: ferramenta para a geração do arquivo de definição de API no formato openAPI, com a possibilidade de expor uma interface Swagger para visualização;
- **Dependências de Compilação**: [lombok](#) e [mapstruct](#) para minimizar métodos verbosos e repetitivos no código fonte. Definem macros que injetam códigos como getters e construtores no código intermediário antes de compilação para instruções bytecode da JVM.

São utilizados dois serviços de apoio: o banco de dados orientado a documentos [MongoDB](#) e o broker de mensagens em fila [RabbitMQ](#). Esses serviços estão definidos no arquivo *docker-compose.yaml* da aplicação, que facilita a rápida configuração do ambiente.

Como já citado, [Docker](#) é utilizado como ferramenta para geração e instância de imagens em [containers](#) dentro do Sistema Operacional. O arquivo Dockerfile do repositório define os passos para *build* da aplicação, que é executado a cada novo commit na *branch main* do repositório remoto no GitHub. Essa é a tecnologia chamada de [workflows](#) da plataforma da Microsoft que permite a criação de esteiras orientadas a eventos.

Com a configuração atual do repositório, sempre que um *commit* é detectado, um processo é iniciado em uma máquina da infraestrutura do GitHub, como uma implementação de [Function-as-a-Service \(FaaS\)](#). Esse processo autentica-se no [DockerHub](#) (repositório de imagens Docker), executa as instruções do arquivo Dockerfile para gerar a imagem, e publica a imagem resultante no *registry*.

Por ser uma aplicação com escopo fechado e construída exclusivamente em Java, o processo de construção das imagens é dividido em somente dois estágios: o *build* e a aplicação. Utilizam-se imagens-base oficiais do Gradle e do Java, respectivamente. Ambas são construídas sobre a distribuição Linux Alpine, a opção mais leve adequada para esta aplicação, que é essencialmente monolítica e não requer dependências robustas adicionais. A única dependência externa declarada para o sistema operacional é o binário [amqp-publish](#), utilizado no script para simulação de carga na fila.

Ambiente de desenvolvimento

Por mais que a aplicação execute em sistemas simples como o Alpine Linux, fato provado na execução da aplicação a partir do binário base na imagem Docker. É uma tarefa um tanto quanto complicada usar um ambiente minimalista desse para desenvolvimento, visto que IDEs, softwares de acesso a Banco de Dados e navegadores precisam de dependências que seriam de responsabilidade do usuário instalar na distribuição.

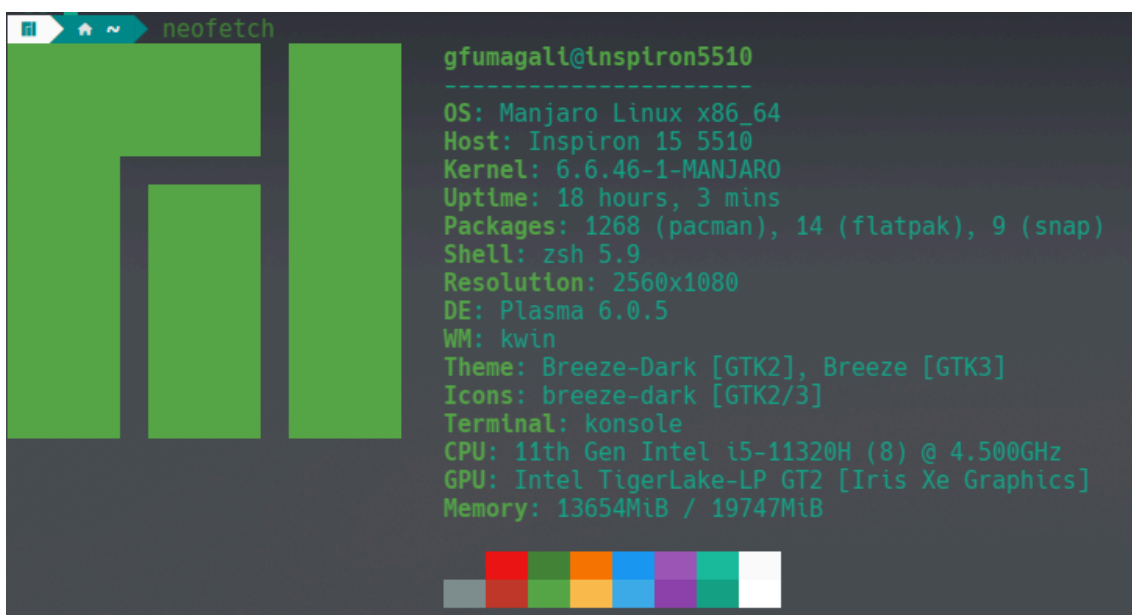


Figura 2. Especificações da máquina de desenvolvimento

A máquina utilizada para o desenvolvimento está representada na Figura 2. Ela é equipada com a distribuição Manjaro, baseada em Arch Linux, e possui um processador Intel x86 da geração de 2021, juntamente com 24 GB de memória RAM em single-channel, compartilhada com a placa gráfica. A IDE utilizada foi o IntelliJ da JetBrains em sua versão ultimate, uma excelente aplicação voltada a Java/Kotlin que traz funcionalidades avançadas como a leitura e escrita no Banco de Dados diretamente da interface, robusto sistema de análise de código, visualização de diagrama de classes, entre outras ferramentas úteis no desenvolvimento.

Diagrama de implantação

Esta seção apresenta uma proposta de implantação que não foi implementada durante a semana de desenvolvimento. O diagrama, ilustrado na Figura 3, propõe duas frentes distintas na infraestrutura: a primeira lida com a geração da imagem a ser instanciada na máquina hospedeira, e a segunda com o método de acesso pelos usuários.

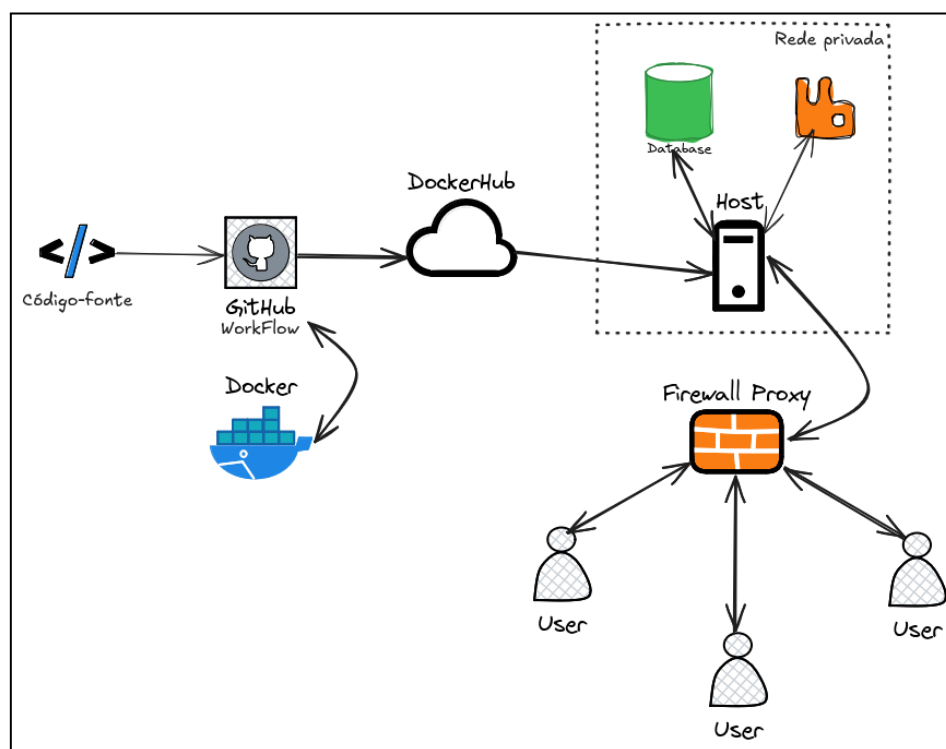


Figura 3. Diagrama gráfico de implantação

O fluxo se inicia com a configuração do workflow mencionado anteriormente, que disponibiliza a imagem mais recente no DockerHub. Em um cenário de produção, seria ideal utilizar um registro privado de imagens.

A máquina *host* acessa a versão estável mais recente da aplicação e a expõe em uma porta específica, acessível externamente por um *proxy* que atua como *firewall*. Como a API não possui mecanismos de autenticação (como chaves de API ou tokens), um proxy, como o serviço oferecido pela CloudFlare, poderia ajudar a prevenir ataques comuns, como negação de serviço e rotinas de enumeração.

Além disso, os serviços de apoio seriam acessíveis somente pela rede do *host*, evitando que atacantes externos acessem informações do estado da aplicação armazenadas no banco de dados. É importante configurar regras de *firewall* no nível do sistema operacional para evitar exposições desnecessárias.

A aplicação segue o princípio *Stateless*, o que significa que não armazena nenhum estado em memória durante sua execução. Isso permite a criação de uma

infraestrutura baseada em *Kubernetes*, por exemplo, para gerenciar múltiplas instâncias da aplicação e atender à demanda com escalonamento horizontal. Nesse cenário, após o *proxy*, um balanceador de carga seria responsável por direcionar as requisições conforme o número de containers instanciados. Na prática, seria necessário migrar as informações de bloqueio para evitar conflitos quando mensagens repetidas são processadas simultaneamente em diferentes instâncias. Com essa melhoria, a solução apresentada se tornaria viável.

Diagrama da arquitetura

A arquitetura do micro serviço foi projetada com base no modelo Repository, Service e Controller, derivado do clássico [padrão de projeto MVC](#), conforme ilustrado na Figura 4. O modelo possui camadas bem definidas e a topologia é a seguinte:

- **Controller:** a camada mais externa da aplicação, que fornece a interface com usuários externos por meio de uma API REST;
- **Service:** lida com regras de negócio, implementando métodos que disparam casos de uso da aplicação. É a camada que recebe as requisições do Controller;
- **Repository:** interface entre o Service e o Banco de Dados, fornece esquemas de leitura e escrita no SGBD. Utiliza um ORM do *Spring Framework*, similar ao JPA, só que para dados em formato de objetos, gerando consultas e operações de atualização;
- **Listener:** classe responsável por se conectar a um tópico de fila e encaminhar mensagens ao Service. No micro serviço, encaminha as mensagens para o caso de uso que persiste novos registros de venda.

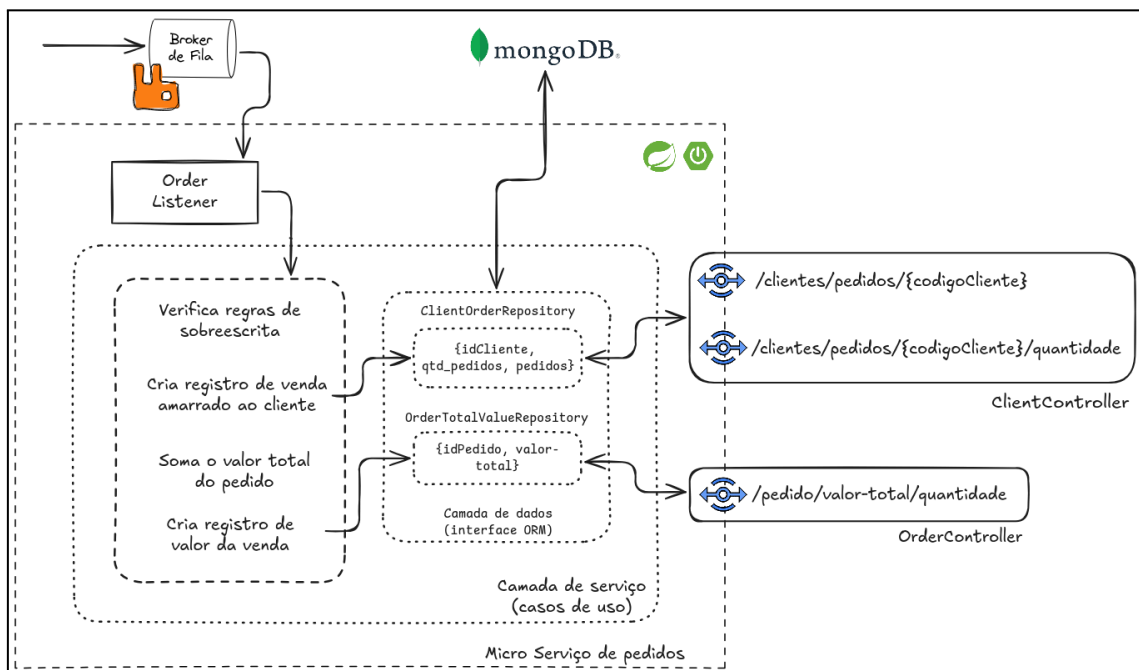


Figura 4. Diagrama da arquitetura da aplicação

Repara-se que não há métodos intermediários que processam as requisições de acesso vindo dos *Controllers* depois de recuperar os registros no repositório. Esse é um comportamento esperado da arquitetura projetada. Uma vez que os dados são consumidos de uma fila, é passível de ceder prioridade para operações de leitura e modelar os dados de modo que qualquer que seja o processamento adicional, estes sejam feitos em momentos de escrita. A próxima seção aborda com mais detalhes essa modelagem.

Modelagem da Base de Dados

É fatídico afirmar que este tópico aborda as decisões mais importantes no projeto de um software, que se propagam em como o software é construído. No micro serviço, foi escolhido um Sistema de Gerenciamento de Banco de Dados (SGBD) orientado a documentos, levantando a questão: por que não utilizar um banco de dados relacional, como o PostgreSQL?

Embora fosse possível utilizar um modelo relacional com duas tabelas, uma para Pedidos e outra para Itens, como ilustrado na Figura 5, surgem desafios relacionados à normalização dos dados em tabelas, especialmente considerando que o preço do produto não é garantido. Assim, para cada “lápiz” recebido, não é possível pressupor que existirá tupla correspondente a esse exato registro na tabela Itens, uma vez que o preço do produto varia, podemos enquadrá-lo como chave candidata. Portanto, pode ser tratada de duas diferentes formas:

- 1) **Relacionamento n:1:** Cada Item pertence a um único pedido, e o preço não é usado como chave identificadora do Item. Isso pode resultar em redundância na base de dados, configurando uma forma de desnormalização;
- 2) **Relacionamento n:n:** O preço é usado como identificador da tupla. Nesse caso, a cada inserção, seria necessário realizar uma busca na tabela Itens para garantir a unicidade de cada item de cada pedido.

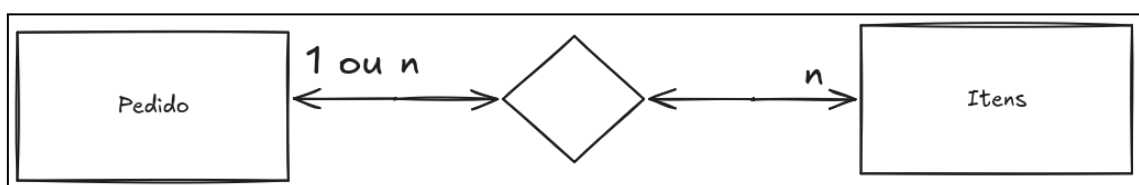


Figura 5. Ilustração do relacionamento entre pedidos e itens.

É difícil afirmar algo sobre performance nessa escala de software. Mas em ambos os casos, por mais que índices praticamente em tempo constante verifiquem a unicidade de <Item, preço> (usando tabela *hash*), é inevitável que haja pelo menos uma operação de junção de chave primária com chave estrangeira. Esse processamento ocorreria em tempo de leitura, um cenário que se pretende evitar.

Nesse contexto, em um software com três consultas necessárias, um Banco de Dados orientado a documentos, com agregados orientado às consultas se torna interessante. A escolha de mapear os dados em agregados foi motivada pelas duas principais classes de consultas requisitadas:

- 1) **Quantidade de Pedidos e Listagem de Pedidos de um Cliente:** Agregado contendo informações sobre todos os pedidos associados à chave primária do cliente;
- 2) **Valor Total de um Pedido:** Agregado que armazena a soma total identificada para cada código de pedido.

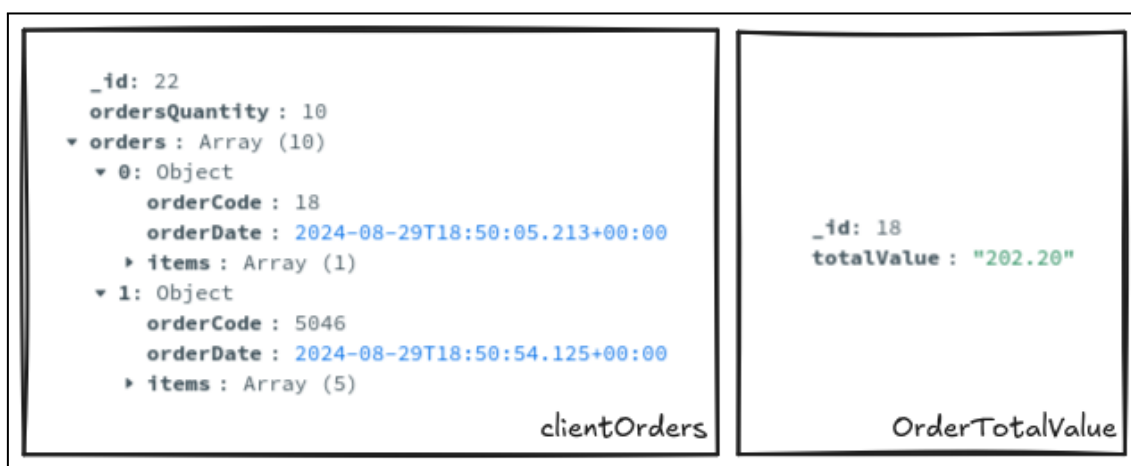


Figura 6. Modelagem orientada a documentos.

A Figura 6 ilustra a modelagem das duas coleções implementadas:

- **Esquema 1:** Resolve o problema da quantidade e listagem de pedidos. Permite a implementação de paginação no *endpoint* de listagem, pois o número de pedidos aumenta ao longo do tempo. A quantidade total de pedidos pode ser facilmente obtida a partir do mesmo agregado de informações.
- **Esquema 2:** Fornece as informações necessárias para calcular o valor total do pedido.

Além das três consultas especificadas, a implementação do sistema de sobrescrita aumenta a carga no banco de dados, que precisa verificar se cada pedido já foi processado anteriormente. Essa operação utiliza um índice de unicidade no campo *orderCode*.

É importante notar que o *array* de pedidos, que é o maior dado potencial da aplicação, nunca é carregado completamente na memória do micro serviço. As operações de atualização enviam apenas o pedido a ser adicionado ao servidor do banco de dados. As operações de leitura utilizam fatias desse *array* para recuperar as informações do usuário em formato de página.

Plano de trabalho

O plano de trabalho previa 38 horas a serem distribuídas ao longo de uma semana. No geral, o plano foi bem-sucedido, com as exceções listadas na tabela 1.

Tabela 1. Tarefas com estimativas divergentes

Tarefa	Prevista	Utilizada	Motivo
Modelagem do Banco de Dados	1,5	4	A complexidade para evitar o carregamento do agregado na memória e o planejamento detalhado foram mais trabalhosos do que o previsto.
Geração de imagens Docker e upload no DockerHub	6	4	O risco da tarefa, dado meu nível de experiência, foi estimado alto. Contudo, a documentação assertiva do GitHub facilitou e acelerou a tarefa.
Testes unitários e de integração.	4	8	A aplicação sofreu mudanças durante o desenvolvimento, resultando em mais horas gastas para ajustar e criar os testes. Por falta de tempo, não foi possível implementar os testes de integração.

O restante da estimativa cobriu adequadamente as subtarefas implícitas, que envolvem o fluxo de: 1) planejamento da tarefa; 2) execução; 3) testes funcionais. As tarefas foram estimadas com base em um fator de risco associado à experiência do desenvolvedor.

Por exemplo, a tarefa de "Documentação da API" foi estimada com um tempo justo, uma vez que eu já a executei diversas vezes em outros projetos, reduzindo a probabilidade de problemas inesperados. Em contraste, a tarefa de "Geração de Imagens Docker e Upload no DockerHub" teve uma estimativa mais alta devido à minha pouca experiência, o que elevou o fator de risco para cobrir tempos de estudo e testes. Essa abordagem de estimativa pode, por vezes, resultar em uma alocação de tempo superior ao ideal, mas, a favor de evitar resultados instáveis ou imaturos.

Testes funcionais

Para os testes funcionais, foi gerada uma carga de testes cujos comandos estão detalhados na Figura 7 e na Tabela 2. Esses comandos abordam casos típicos de mensagens, bem como casos de repetição, sobrescrita de pedidos e dados incompletos.

Tabela 2. Dados para execução de testes funcionais

codigoPedido	codigoCliente	Itens	Valor total
1	1	"batata" - 10un - R\$ 1,90 "cenoura" - 23un - R\$ 2,00	R\$ 75,00
2	1	"lápis" - 2un - R\$1,20 "borracha" - 5un - R\$ 7,20	R\$ 38,40
3	2	"lápis" - vazio - R\$1,00 "caneta" - 5un - R\$ 7,20	Inválido
3	2	"lápis" - 2un - R\$1,00 "caneta" - 5un - R\$ 7,20	R\$ 38,00
4	2	"caderno" - 1un - R\$23,99 "caneta" - 5un - R\$ 7,20	R\$ 59,99
5	2	"fiat uno" - 1un - R\$31945,35 "alface" - 12un - R\$ 2,20 "iogurte" - 3un - R\$ 5,10 Mensagem enviada 3x	R\$ 31987,05
1	1	"batata" - 10un - R\$ 1,90 "cenoura" - 23un - R\$ 1,00	R\$ 42,00
4	3	"coca-cola" - 3un - R\$ 4,10	R\$ 12,30

```

# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 1, "codigoCliente": 1, "itens": [{"produto": "batata", "quantidade": 10, "preco": 1.90}, {"produto": "cenoura", "quantidade": 23, "preco": 2.00}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 2, "codigoCliente": 1, "itens": [{"produto": "lapis", "quantidade": 2, "preco": 1.20}, {"produto": "borracha", "quantidade": 5, "preco": 7.20}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 3, "codigoCliente": 2, "itens": [{"produto": "lapis", "quantidade": "", "preco": 1}, {"produto": "caneta", "quantidade": 5, "preco": 7.20}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 3, "codigoCliente": 2, "itens": [{"produto": "lapis", "quantidade": 2, "preco": 1}, {"produto": "caneta", "quantidade": 5, "preco": 7.20}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 4, "codigoCliente": 2, "itens": [{"produto": "caderno", "quantidade": 1, "preco": 23.99}, {"produto": "caneta", "quantidade": 5, "preco": 7.20}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 5, "codigoCliente": 2, "itens": [{"produto": "fiat uno", "quantidade": 1, "preco": 31945.35}, {"produto": "alface", "quantidade": 12, "preco": 2.2}, {"produto": "iogurte", "quantidade": 3, "preco": 5.1}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 5, "codigoCliente": 2, "itens": [{"produto": "fiat uno", "quantidade": 1, "preco": 31945.35}, {"produto": "alface", "quantidade": 12, "preco": 2.2}, {"produto": "iogurte", "quantidade": 3, "preco": 5.1}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 5, "codigoCliente": 2, "itens": [{"produto": "fiat uno", "quantidade": 1, "preco": 31945.35}, {"produto": "alface", "quantidade": 12, "preco": 2.2}, {"produto": "iogurte", "quantidade": 3, "preco": 5.1}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 1, "codigoCliente": 1, "itens": [{"produto": "batata", "quantidade": 10, "preco": 1.90}, {"produto": "cenoura", "quantidade": 23, "preco": 1.00}]}'
# curl -X POST http://localhost:8080/pedidos -H 'Content-Type: application/json' -d '{"codigoPedido": 4, "codigoCliente": 3, "itens": [{"produto": "coca-cola", "quantidade": 3, "preco": 4.10}]}'

```

Figura 7. Sequência de comandos para carga da aplicação

Valor total dos pedidos

```

Unset
for i in {1..5}
do
    valor=$(curl -s "http://localhost:8080/pedido/valor-total/$i")
    echo "Pedido $i - R$ $valor"
done

```

```
done
- * - * - * - * - * - * - * - * - * - * - * - * - * - * - * - * - * - * - * - * - * - *
Pedido 1 - R$ 42.00
Pedido 2 - R$ 38.40
Pedido 3 - R$ 38.0
Pedido 4 - R$ 59.99
Pedido 5 - R$ 31987.05
```

A execução do *script* mostra que, além da soma esperada, o valor recebido possui no máximo duas casas decimais. Observa-se também que:

- O valor do Pedido 1 corresponde ao valor da última mensagem enviada, que sobrescreve a primeira;
- A replicação da mensagem para o Pedido 5 não afetou o valor final;
- A tentativa de sobrescrição do pedido feito pelo cliente 2 com uma mensagem em nome do cliente 3 foi ignorada.

Esses comportamentos são confirmados pelos logs da aplicação:

- Figura 8 mostra o log de erro referente à não permissão de sobrescrita.
- Figura 9 captura o erro de validação na terceira mensagem.

```
2024-08-30T10:07:36.920-03:00 ERROR 384247 --- [btg-micro-servico-pedidos] [ntContainer#0-1] m.g.b.listener.ListenerErrorHandler : Error processing
message GenericMessage [payload=OrderDTO(orderCode=4, clientCode=3, items=[ItemDTO(product=coca-cola, quantity=3, price=4.10)]),
headers={amqp_receivedDeliveryMode=NON_PERSISTENT, amqp_receivedRoutingKey=pedidos, amqp_deliveryTag=13, amqp_consumerQueue=pedidos, amqp_channel=Cached
Rabbit Channel: AMQChannel(amqp://user@127.0.0.1:5672/,1), conn: Proxy@29f97a17 Shared Rabbit Connection: SimpleConnection@411a9c73
[delegate=amqp://user@127.0.0.1:5672/, localPort=39094], amqp_redelivered=false, id=d8514f99-968d-1ee9-9f64-64e128259f2b, amqp_consumerTag=amq
.ctag-sJvjsnUp8ipmR3ffpggF3A, amqp_lastInBatch=false, contentType=application/json, timestamp=1725023256920]} with 108 bytes
2024-08-30T10:07:36.920-03:00 ERROR 384247 --- [btg-micro-servico-pedidos] [ntContainer#0-1] m.g.b.listener.ListenerErrorHandler : Caused by:
java.lang.IllegalArgumentException Create breakpoint : Código de pedido já existe para o cliente
```

Figura 8. Log de erro correspondente a não permissão de sobrescrita

```
2024-08-30T09:59:15.623-03:00 ERROR 384247 --- [btg-micro-servico-pedidos] [ntContainer#0-1] m.g.b.listener.ListenerErrorHandler : Error processing
message GenericMessage [payload=OrderDTO(orderCode=3, clientCode=2, items=[ItemDTO(product=lapis, quantity=0, price=1), ItemDTO(product=caneta,
quantity=5, price=7.2)]), headers={amqp_receivedDeliveryMode=NON_PERSISTENT, amqp_receivedRoutingKey=pedidos, amqp_deliveryTag=6,
amqp_consumerQueue=pedidos, amqp_channel=Cached Rabbit Channel: AMQChannel(amqp://user@127.0.0.1:5672/,1), conn: Proxy@29f97a17 Shared Rabbit Connection:
SimpleConnection@411a9c73 [delegate=amqp://user@127.0.0.1:5672/, localPort=39094], amqp_redelivered=false, id=bfd18edf-53c0-c37e-577c-5f7ed4b1b68f,
amqp_consumerTag=amq.ctag-sJvjsnUp8ipmR3ffpggF3A, amqp_lastInBatch=false, contentType=application/json, timestamp=1725022755623]} with 156 bytes
2024-08-30T09:59:15.632-03:00 ERROR 384247 --- [btg-micro-servico-pedidos] [ntContainer#0-1] m.g.b.listener.ListenerErrorHandler : Caused by:
validation error: Could not resolve method parameter at index 0 in public void me.gfumagali.btgpedidos.listener.OrderListener.receive(me.gfumagali
.btgpedidos.model.dto.OrderDTO): 1 error(s): [Field error in object 'in' on field 'items[0].quantity': rejected value [0]; codes [Positive.in.items[0]
.quantity,Positive.in.items.quantity,Positive.items[0].quantity,Positive.items.quantity,Positive.quantity,Positive.int,Positive]; arguments [org
.springframework.context.support.DefaultMessageSourceResolvable: codes [in.items[0].quantity,items[0].quantity]; arguments []; default message [items[0]
.quantity]]; default message [Quantity must be positive]]
```

Figura 9. Log de erro capturado em uma falha de validação

Quantidade de pedidos por cliente

Para verificar a quantidade de pedidos por cliente, foi utilizado o seguinte *script*:

```
Unset
for i in {1..4}
do
    response=$(curl -s -w "%{http_code}" -o response.txt
"http://localhost:8080/cliente/pedidos/$i/quantidade")

    http_code=$(tail -n1 <<< "$response")

    if [ "$http_code" -eq 404 ]; then
        message=$(<response.txt)
        echo "Cliente $i - $message"
    else
        qtd=$(<response.txt)
        echo "Cliente $i - $qtd pedidos realizados"
    fi
    rm response.txt
done

-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+-+
Cliente 1 - 2 pedidos realizados
Cliente 2 - 3 pedidos realizados
Cliente 3 - Cliente não encontrado
Cliente 4 - Cliente não encontrado
```

Este *script* envia uma requisição para o servidor, salva o conteúdo da resposta em um arquivo e exibe na saída padrão do terminal o código HTTP retornado. Se a resposta for código 200 (OK), a quantidade de pedidos é formatada para melhor visualização. Se a resposta for 404 (NOT FOUND), a mensagem retornada no corpo da resposta é exibida.

Com isso, podemos confirmar:

- **Cliente 1:** Apresenta 2 pedidos realizados, onde uma das três mensagens enviadas foi cancelada por sobrescrição.
- **Cliente 2:** Tem 3 pedidos realizados, com mensagens repetidas não afetando o comportamento esperado.
- **Cliente 3:** Sua mensagem foi cancelada por tentar sobrescrever um pedido de outro cliente e não aparece na API.
- **Cliente 4:** Nunca fez um pedido, portanto, não aparece na API.

Lista de pedidos por cliente

Para obter a lista de pedidos por cliente, foi utilizado o seguinte *script*:

```
for i in {1..4}
```

do

```
if [ "$http_code" -eq 404 ]; then
    message=$(
```

[illegible]

```

Cliente 1 - {
  "content": [
    {
      "codigoPedido": 1,
      "dataPedido": "2024-08-30T10:04:54.57",
      "itens": [
        {
          "produto": "batata",
          "quantidade": 10,
          "preco": 1.90
        },
        {
          "produto": "cenoura",
          "quantidade": 23,
          "preco": 1.00
        }
      ]
    },
    {
      "codigoPedido": 2,
      "dataPedido": "2024-08-30T09:58:09.414",
      "itens": [
        {
          "produto": "lapis",
          "quantidade": 2,
          "preco": 1.20
        },
        {
          "produto": "borracha",

```

```

        "quantidade": 5,
        "preco": 7.2
    }
]
}
],
... informações de paginação
}
Cliente 2 - {
  "content": [
    {
      "codigoPedido": 3,
      "dataPedido": "2024-08-30T09:59:59.442",
      "itens": [
        {
          "produto": "lapis",
          "quantidade": 2,
          "preco": 1
        },
        {
          "produto": "caneta",
          "quantidade": 5,
          "preco": 7.2
        }
      ]
    },
    {
      "codigoPedido": 4,
      "dataPedido": "2024-08-30T10:00:57.719",
      "itens": [
        {
          "produto": "caderno",
          "quantidade": 1,
          "preco": 23.99
        },
        {
          "produto": "caneta",
          "quantidade": 5,
          "preco": 7.2
        }
      ]
    },
    {
      "codigoPedido": 5,
      "dataPedido": "2024-08-30T10:36:10.142",
      "itens": [

```

```
{
  "produto": "fiat uno",
  "quantidade": 1,
  "preco": 31945.35
},
{
  "produto": "alface",
  "quantidade": 12,
  "preco": 2.2
},
{
  "produto": "iogurte",
  "quantidade": 3,
  "preco": 5.1
}
]
},
... informações de paginação
}
Cliente 3 - Cliente não encontrado
Cliente 4 - Cliente não encontrado
```

A saída acima replica os dados especificados na tabela 1, demonstrando o correto funcionamento da API. Observa-se que, nas mensagens repetidas, a data do pedido confirma que a mensagem considerada é sempre a última enviada, mesmo que o conteúdo seja o mesmo.